Relatório de continuidade — desempenho do ASTRO BOT no AnyPS5

Data: 24/10/2026

Jogo: ASTRO BOT — `PPSA21567`, content `01.018.000`

Runtime: `app.exe` da v0.1.3 re-relinkado + libs da `main` (`a0fb7c26a7969d0ae2b18aee923e43a30d133898`)

Escopo: continuidade do porte depois da correção do crash de GPU; investigação de desempenho e de custo de CPU.

Este documento é o sucessor de RELATORIO_PORTE_ASTRO_BOT_ANYPS5.md,

que cobre o crash de alocação e o procedimento de build. Aqui está só o que veio depois.

---

1. Estado em uma página

- **O crash está resolvido e não voltou.** Nenhuma das rodadas desta fase teve `what()`, `VK_ERROR`,

`OUT_OF_DEVICE_MEMORY`, `device lost` ou `abort`. O jogo chega ao título (`title_controller_ship`)

e ao `ps_logo` em todas as configurações testadas.

- **O gargalo não é a GPU.** Nos perfis, a GPU fica ociosa entre 75% e 95% do tempo

(`idle gaps` de 1,2 a 12,5 s por frame) e o blit de apresentação custa **0,13 ms por frame**.

- **O gargalo é CPU e espera de label.** Os três queue workers ficam em 88-103% de ocupação e as

filas `0x40`/`0x48` passam a maior parte do intervalo dentro de `WAIT_REG_MEM`, esperando

labels de 0,4 a 1,4 s (p50) que a **fila 0** deveria produzir.

- **Maior ganho já obtido:** `APS5_HOST_IMPORT_MIB=6144` (de 403 ms para 81 ms de p50 nas rodadas

da fase anterior). A motivação está documentada na dívida técnica do projeto: um label gravado em

memória guest sem host import espera todo o trabalho gravado e é escrito pela CPU.

- **Ganho novo e acionável desta fase:** `APS5_PIPELINED_DRAWS=32` reduz o custo de CPU por commit

de 594 µs para 121 µs (e o `vectors` de 1.218 µs para 172 µs) **sem piorar** a latência das esperas.

- **Causa raiz em aberto:** por que a fila 0 demora a produzir os labels que as filas de compute

esperam — incluindo o caso em que a própria fila 0 passa 2,7 s por 10 s esperando.

- **Upscaling (FSR/DLSS/PSSR) foi descartado como estratégia de desempenho** com base em números:

atuaria no blit de apresentação, que custa 0,13 ms por frame.

---

2. Ambiente

Item	Valor
Sistema	Windows 11 IoT Enterprise LTSC 10.0.26100, 64 bits
CPU	Intel Core i7-7700, 4 núcleos / 8 threads
RAM	16 GB
GPU	AMD Radeon RX 9060 XT, 16 GB, driver `32.0.32015.7029`
Emulador	AnyPS5 v0.1.3 (runtime) + libs da `main` `a0fb7c2`
Fonte	`D:\AnyPS5-main` (shallow, commit `a0fb7c2`, árvore limpa)

Item	Valor
Build	`D:\AnyPS5-main\build`, Ninja + GCC 15.2.0, `--target libs`
Libs em uso	`AnyPS5-AstroBot\work\libs` — 375 arquivos, 294 MiB
`libSceAgcDriver.prx`	17.104.761 bytes, SHA-256 `133AD9421B5DED833066E18DEC86B4F2D3DABFD36DD43C1F5EDB22D6F1EB2170`
`app.exe`	276.375.552 bytes, SHA-256 `60740BB516214015CDEDE538B71B1159D021083911F7A5459C300251C78F2F5D`
Cache de shaders	`work\shader_cache\8dd259103f14b7f7` quente (1.600+ acertos por intervalo, 0 misses)

---

3. O que já estava pronto (fase 1)

Resumo do relatório anterior, para não repetir:

1. A v0.1.3 abortava ao criar um heap mirror de 3.388.997.632 bytes, acima do `maxMemoryAllocationSize` de 2 GiB do driver AMD.
2. A `main` corrige em PRs #7292 (importa faixas em pedaços) e #7647 (heap mirror só nas páginas escritas). O conjunto **completo** de libs foi compilado e copiado para `work\libs`.
3. Resultado: o jogo passa do crash, carrega o título e apresenta frames. Backup limpo da v0.1.3 em `work\libs.v0.1.3.full.bak`.

O que a fase 1 explicitamente **não** cobriu: gameplay, áudio, controles, desempenho e a reconstrução de `app0`.

---

4. Harness de medição (novo)

Antes desta fase as comparações eram manuais, com `Start-Process` digitado a cada rodada.

Arquivo	Função
`bench\run-case.ps1`	roda um caso com env controlado, mata processos órfãos, aplica timeout, grava logs e chama o relatório
`bench\report.ps1`	extrai milestones, budget de import, `[present]`, `[packets]`, `[draw]`, `[gputime]`, timeouts e a distribuição de latência das esperas
`bench\README.md`	uso, significado de cada linha e a tabela de resultados

Casos nomeados: `baseline`, `import8`, `labels`, `flush`, `prio`, `prio-rt`.

Qualquer variável extra entra por `-Env "NOME=VALOR;NOME2=VALOR2"`.

Saída por rodada: `bench\runs\<caso>-<timestamp>\` com `out.log`, `err.log`, `summary.txt`, `aps5-summary.jsonl` e `aps5-bench-*.json.gz`.

Limitação conhecida: todas as rodadas desta fase medem carregamento e tela de título (nenhum input é injetado). Os números servem para comparar configurações, não para reportar FPS.

---

5. Rodadas executadas nesta fase

#	Rodada	Configuração	Duração	Esperas (n)	Espera p50	Espera p90	Espera total	Observação
1	`baseline-105948`	host import 6144, sem perfil	90 s	—	—	—	—	p50 do frame 111,8 ms; p90 416,4 ms; 1 stall de 35,5 s no load

#	Rodada	Configuração	Duração	Esperas (n)	Espera p50	Espera p90	Espera total	Observação
2	`labels-110221`	+ `APS5_PROFILE_*` + `APS5_TRACE_GPU`	120 s	136	411,4 ms	1.321,2 ms	131,4 s	referência do A/B
3	`labels-110523`	#2 + `APS5_LABEL_SUBMIT_NOW=1`	120 s	126	354,9 ms	2.624,3 ms	134,0 s	**sem efeito**
4	`labels-110938`	#2 + `APS5_PIPELINED_DRAWS=0`	120 s	80	**1.374,7 ms**	3.357,3 ms	145,8 s	**inline é pior**
5	`labels-111144`	#2 + `APS5_PIPELINED_DRAWS=32`	120 s	147	448,6 ms	1.326,5 ms	**121,7 s**	**melhor custo de CPU**

Custo de commit medido em cada configuração:

Configuração	commits / 10 s	µs por commit	`vectors` µs por packet
`APS5_PIPELINED_DRAWS=0`	sem pipeline	—	—
default (8)	3.222	593,7	1.218,2
`APS5_PIPELINED_DRAWS=32`	5.445	**120,7**	**171,7**

> Rodadas com perfil são 3-10× mais lentas que a real: `APS5_PROFILE_DRAW` instrumenta cada escopo e `APS5_TRACE_GPU` imprime duas linhas por espera. Compare colunas, não valores absolutos.

---

6. Leitura dos dados

6.1 A GPU não é o limite

`[present]`: GPU ocupada 40-110 ms por frame, com `idle gaps` de 1,2 a 12,5 s. No run de diagnóstico (`work\perf.err.log`), o tempo de GPU por frame é:

Classe	Tempo por frame
shadow-publish	48,5 ms (247 MiB)
storage-writeback	26,6 ms (342 MiB)
storage-upload	4,3 ms
draw	2,7 ms
present-blit	**0,13 ms**

Consequência direta: qualquer upscaler (FSR1/EASU/FSR2) atuaria nos 0,13 ms do blit. DLSS é inviável (exige RTX, motion vectors e jitter que o título não fornece). O PSSR da Sony (`scePsmlMfsr`) está stubado no emulador e o ASTRO BOT não o chama nos logs. A documentação também confirma que **não existe override global de resolução de render** no AnyPS5 (`docs/user/USAGE.md`), então reduzir os render targets exigiria mudança invasiva e sem retorno enquanto a GPU estiver ociosa.

6.2 Onde o tempo de CPU vai

- `[draw] driver phases`: o maior campo é `vectors` (81 µs limpo, 1.218 µs com perfil). Ele **não** é montagem de vértices: no código é o marcador de fase que envolve `drawWriteRanges`, a enfileiração no pipeline e a espera por vaga.
- `[packets] queue 0x0`: `WAIT_REG_MEM_64` aparece com 2,7 s por 10 s no run com depth 32, e a

estatística mostra **532 esperas satisfeitas "same queue"** na janela.

- `[present]` no run com perfil: `Driver.Packet.memory_wait` é a maior fase por frame.

6.3 Hipóteses testadas e refutadas

Hipótese	Teste	Resultado
Os labels atrasam porque o deadline de flush (1 ms) é longo	`APS5_LABEL_SUBMIT_NOW=1`	sem efeito (p50 355 vs 411 ms; total 134 vs 131 s) — **refutada**
O committer serializado atrasa os labels	depth 8 (594 µs/commit) vs depth 32 (121 µs/commit)	latência igual com 5× menos CPU — **refutada**
Desligar o pipeline ajuda por remover fila	`APS5_PIPELINED_DRAWS=0`	pior (p50 1.375 ms) — **refutada**

6.4 O que sobrou

Os mesmos quatro labels aparecem em todas as rodadas, sempre nas filas `0x40`/`0x48`:

```
0x400200040   0x4002012c0   0x400202180   0x400202f80
```

Todos com `function 3 ref 0x1`, valor esperado `0x1`, e o histórico de escrita mostrando um `RELEASE_MEM` anterior para o mesmo endereço. Ou seja: **o store existe, mas chega tarde**, e a própria fila 0 também espera (2,7 s/10 s). O próximo passo é identificar em que ponto do stream da fila 0 esse `RELEASE_MEM` é gravado e o que o atrasa.

---

7. Como reproduzir

```
# baseline comparável (sem perfil)
powershell -NoProfile -ExecutionPolicy Bypass -File bench\run-case.ps1 -Case baseline -Seconds 90

# diagnóstico de espera (perfil + trace)
powershell -NoProfile -ExecutionPolicy Bypass -File bench\run-case.ps1 -Case labels -Seconds 120 -Profile

# rodada específica
powershell -NoProfile -ExecutionPolicy Bypass -File bench\run-case.ps1 -Case labels -Seconds 120 -Profile -Env "

# reprocessar
powershell -NoProfile -ExecutionPolicy Bypass -File bench\report.ps1 -RunDir bench\runs\<pasta>
```

---

8. Inventário

Runtime e jogo

- `work\app.exe`, `work\libs`, `work\app0`, `work\shader_cache`
- Backups: `work\libs.v0.1.3.full.bak` (rollback recomendado), `work\libs.v0.1.3.bak` (parcial, não usar)
- `work\app_rebuilt.exe` — re-relink feito em 23/10 20:46, **nunca executado**; hash `B991CC96…`
- `work\run-astro-optimized.cmd` — atalho atual, só define `APS5_HOST_IMPORT_MIB=6144`

Medição

- `bench\run-case.ps1`, `bench\report.ps1`, `bench\README.md`
- `bench\runs\` — 5 rodadas desta fase
- `work\traces-*` — rodadas da fase anterior (12 pastas; `traces-safe` e `traces-safe2` estão vazias)

Documentação

- `RELATORIO_PORTE_ASTRO_BOT_ANYPS5.md` / `.html` / `.pdf` — fase 1 (crash e build)

- `RELATORIO_CONTINUIDADE_PERFORMANCE.md` / `.pdf` — este documento

---

9. Próximos passos, em ordem

1. **Identificar o produtor dos labels** (em andamento): `APS5_TRACE_LABEL=0x400202f80` + `APS5_TRACE_GPU` para ver o `RELEASE_MEM` no momento em que é gravado e a distância até o wait.
2. **Explicar os 2,7 s/10 s de espera dentro da própria fila 0** e as 532 esperas "same queue".
3. **Adotar** `APS5_PIPELINED_DRAWS=32` no `run-astro-optimized.cmd` depois de validar em gameplay; testar 64 como próximo ponto.
4. **Medir gameplay de verdade**: `APS5_PAD_INPUT_FILE` + `AnyPS5-main\tools\pad_input.py` para sair do título e medir 30 s parado em uma fase.
5. **Investigar as recusas de host import** (21,8 GiB recusados nos primeiros 90 s): cada faixa recusada é copiada a cada uso, o que volta a custar CPU.
6. **Carregamento de imagem** (detile/upload síncrono, `LoadLevelResources` de 3,6-10,4 s): medir com `APS5_PROFILE_DRAW` + `APS5_TRACE_BIGBUF` antes de mexer em código; candidato a pool de upload assíncrono, que é isolado do caminho de render.

---

10. Armadilhas registradas

- **Perfil custa caro.** Nunca compare uma rodada com perfil contra uma sem.
- **Timeout de** `WAIT_REG_MEM` **não aborta.** Ele imprime o aviso e continua esperando (`WaitMemory.cpp`); o custo real é a espera inteira, não 1 s.
- `vectors` **não é vértice.** É o catch-all do caminho de commit; ler o valor como "custo de montagem de vértice" leva à conclusão errada.
- **Rodadas no título não representam gameplay.** A cena muda o p50 em 5×.
- **Cache quente.** Todas as rodadas aqui usaram o cache de shaders populado; a primeira execução em máquina limpa é muito mais lenta.
- **Ambiente PowerShell neste harness:** variáveis são definidas por script (`-Env`), porque o shell da sessão não expande `$`.